

Info

Name: Cody (Zhexian) Liu

Assignment Number: 1

Repo Link: <https://github.com/Rorical/CWM-FDI>

Basic Timing

What is the difference you observe between real, user, and sys time?

```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ /usr/bin/time -v time python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000
0.21user 0.00system 0:00.23elapsed 99%CPU (0avgtext+0avgdata 13308maxresident)k
164inputs+0outputs (0major+1945minor)pagefaults 0swaps
  Command being timed: "time python3 matmul_slow.py"
  User time (seconds): 0.21
  System time (seconds): 0.01
  Percent of CPU this job got: 99%
  Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.23
  Average shared text size (kbytes): 0
  Average unshared data size (kbytes): 0
  Average stack size (kbytes): 0
  Average total size (kbytes): 0
  Maximum resident set size (kbytes): 13308
  Average resident set size (kbytes): 0
  Major (requiring I/O) page faults: 0
  Minor (reclaiming a frame) page faults: 2030
  Voluntary context switches: 46
  Involuntary context switches: 5
  Swaps: 0
  File system inputs: 244
  File system outputs: 0
  Socket messages sent: 0
  Socket messages received: 0
  Signals delivered: 0
  Page size (bytes): 4096
  Exit status: 0
ubuntu@ubuntu:~/CWM-FDI/assignment1$ 
ubuntu@ubuntu:~/CWM-FDI/assignment1$ time python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000

real    0m0.234s
user    0m0.224s
sys     0m0.009s
```

The system time is smallest among all the time measurements. The real time is the longest because it measures the total amount of time the program spend. It is the sum of user and sys time.

Which summary statistics would you report: mean, median, or minimum? Why?

```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ for i in 1 2 3 4 5; do /usr/bin/time -f "%e" python3 matmul_slow.py; done
n=128 reps=2 checksum=107389.290000
0.22
n=128 reps=2 checksum=107389.290000
0.22
n=128 reps=2 checksum=107389.290000
0.22
n=128 reps=2 checksum=107389.290000
0.22
n=128 reps=2 checksum=107389.290000
0.22
```

Mean. The mean time of execution represents the expectation of time the program costs, which will be the real and stable amount of time it takes when deployed in large scale.

Reading Counters with perf stat

Report the output.

```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ sudo perf stat -e cycles,instructions python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000

Performance counter stats for 'python3 matmul_slow.py':

      817666898      cycles
      2299661070      instructions          #    2.81  insn per cycle

      0.234871740 seconds time elapsed

      0.224805000 seconds user
      0.009032000 seconds sys
```

It reported 817666898 cycles; 2299661070 instructions with 2.81 instructions-per-cycle ratio

Report the instructions-per-cycle ratio. What does it suggest?

Report cache misses you observe. Why may this matter?

```

ubuntu@ubuntu:~/CWM-FDI/assignment1$ sudo perf stat -r 5 -e cycles,instructions,cache-misses python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000

Performance counter stats for 'python3 matmul_slow.py' (5 runs):

      809901778      cycles
    ( +- 0.65% )
    2295406350      instructions          #    2.83  insn per cycle
    ( +- 0.04% )
      686353      cache-misses
    ( +- 2.75% )

    0.22736 +- 0.00147 seconds time elapsed ( +- 0.64% )

```

The perf shows 2.83 instructions per cycle. It suggests that CPU is efficiently executing multiple instructions in a single cycle. With that value higher, the program is more efficient and execute faster because it has higher CPU utilization.

Cache miss is 686353 times. This means CPU cannot find the data it want in caches, and has to fetch from memory or other places. CPU has three layers of caches that accelerate its memory reads. More cache miss means slower execution due to bandwidth limits between CPU and memory and each memory operation costs more time.

Hotspot discovery with perf record and perf report

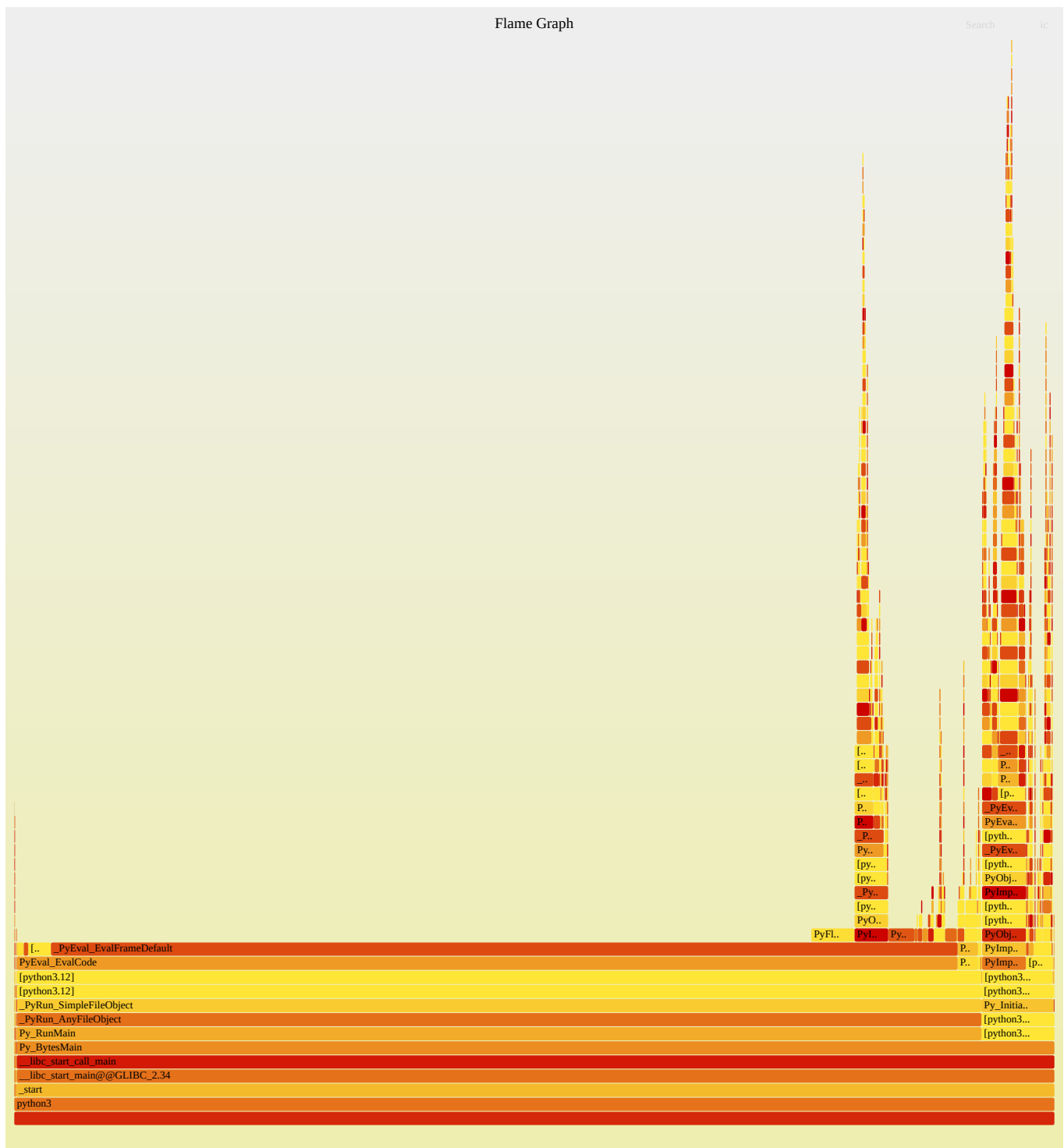
Samples: 919 of event 'cycles:P', Event count (approx.): 811549601			
Overhead	Command	Shared Object	Symbol
75.47%	python3	python3.12	[.] _PyEval_EvalFrameDefault
3.41%	python3	python3.12	[.] PyFloat_FromDouble
3.19%	python3	python3.12	[.] PyLong_FromLong
1.54%	python3	python3.12	[.] _Py_NewReference
0.99%	python3	python3.12	[.] 0x00000000000156483
0.55%	python3	python3.12	[.] 0x0000000000015646f
0.55%	python3	python3.12	[.] PyObject_Malloc
0.44%	python3	python3.12	[.] PyObject_Free
0.44%	python3	[kernel.kallsyms]	[k] __irqentry_text_end
0.31%	python3	[kernel.kallsyms]	[k] strncmp
0.30%	python3	[kernel.kallsyms]	[k] inflate_fast
0.22%	python3	python3.12	[.] 0x00000000000156458
0.22%	python3	python3.12	[.] 0x0000000000015647a
0.22%	python3	python3.12	[.] PyLong_AsDouble
0.22%	python3	python3.12	[.] 0x0000000000021d940
0.22%	python3	python3.12	[.] 0x0000000000021db64
0.22%	python3	python3.12	[.] 0x0000000000021dbc4
0.22%	python3	libc.so.6	[.] __memset_avx2_unaligned_erms
0.22%	python3	python3.12	[.] _PyCode_New
0.22%	python3	python3.12	[.] PyObject_IS_GC
0.20%	python3	[kernel.kallsyms]	[k] clear_page_erms
0.20%	python3	libc.so.6	[.] __memmove_avx_unaligned_erms
0.18%	python3	[kernel.kallsyms]	[k] native_irq_return_iret
0.16%	python3	[kernel.kallsyms]	[k] do_user_addr_fault
0.13%	python3	[kernel.kallsyms]	[k] copy_mc_enhanced_fast_string
0.11%	python3	python3.12	[.] PyLong_AsLongAndOverflow
0.11%	python3	python3.12	[.] 0x0000000000015bbdf
0.11%	python3	python3.12	[.] PyLong_AsLong
0.11%	python3	[kernel.kallsyms]	[k] error_entry
0.11%	python3	python3.12	[.] 0x0000000000016f6d2
0.11%	python3	python3.12	[.] 0x0000000000018b74a
0.11%	python3	python3.12	[.] 0x0000000000021dbf0
0.11%	python3	python3.12	[.] 0x00000000000156e71
0.11%	python3	python3.12	[.] 0x0000000000016a705
0.11%	python3	python3.12	[.] 0x0000000000015647d
0.11%	python3	python3.12	[.] 0x0000000000021d320
0.11%	python3	python3.12	[.] 0x00000000000166e15
0.11%	python3	python3.12	[.] 0x0000000000021ced8
0.11%	python3	python3.12	[.] 0x0000000000021db89
0.11%	python3	python3.12	[.] 0x00000000000156daa
0.11%	python3	python3.12	[.] 0x00000000000159c76
0.11%	python3	[kernel.kallsyms]	[k] alloc_pages_mpol

Which function consumes the largest fraction of samples?

PyEval_EvalFrameDefault consumes 75% of overhead, but it is used for execution of python bytecode, which is not interesting. The second large consumption comes from

PyFloat_FromDouble which means python creating floating point number objects. I guess that in doing the $O(n^3)$ operations to each element of matrix, it create new floating point for each one which slows down the program.

Flame Graphs



a. Which function dominates runtime?

PyEval_EvalFrameDefault

b. Given this dominating call, what are the likely bottlenecks in the matmul_slow() program?

PyEval_EvalFrameDefault is used to execute python codes. This domination means it spent most of time executing pure python code. Python is interpreter based and very slow to execute line by line. The bottleneck is that all the computationally intensive functions are in python. It is

better to use numpy or torch which dispatch the actual computation to C code and optimized hardware.

Propose optimizations for speeding up the execution of `matmul_slow.py`

The matrix is defined as double nested list in the code:

```
Matrix = List[List[float]]
```

The slow code just compute each element that access a in consecutive rows but b in consecutive columns. This is not cache friendly to b because each time the pointer jumps far away to the next row, and cache only store consecutive memory words. The better way is to also access b in consecutive rows, such as taking transpose of it first, then multiple a and b row by row.